

Análise de Desempenho e Confiabilidade em Camadas de Transporte: TCP vs R-UDP

Gabriel Ramos Santos

¹ Curso de Bacharelado em Sistemas de Informação
Universidade Federal do Piauí – UFPI
Campus Senador Helvídio Nunes de Barros

`gabriel.ramos@ufpi.edu.br`

Abstract. *This paper presents a comparative analysis between a file transfer system using native TCP and another using UDP with an implemented reliability layer. The Reliable UDP protocol uses sequence numbers, acknowledgements, timeout, retransmission and checksum validation. Tests were performed in Docker containers under three network conditions using traffic control. The results show that TCP maintains better throughput under packet loss, while the Stop-and-Wait R-UDP implementation is strongly affected by delay and packet loss.*

Resumo. *Este trabalho apresenta uma análise comparativa entre um sistema de transferência de arquivos utilizando TCP nativo e outro utilizando UDP com uma camada de confiabilidade implementada. O protocolo R-UDP utiliza números de sequência, confirmações, timeout, retransmissão e validação por checksum. Os testes foram realizados em containers Docker sob três condições de rede utilizando controle de tráfego. Os resultados mostram que o TCP mantém melhor vazão sob perda de pacotes, enquanto a implementação R-UDP baseada em Stop-and-Wait é fortemente afetada pelo atraso e pela perda.*

1. Introdução

O objetivo deste trabalho foi implementar e comparar dois sistemas de transferência de arquivos: um utilizando sockets TCP e outro utilizando UDP com uma camada de confiabilidade implementada na aplicação. O trabalho exigia controle de erros, controle de fluxo, cabeçalho personalizado, simulação de condições adversas de rede no Docker e análise estatística dos resultados.

O TCP foi utilizado como referência por já possuir mecanismos nativos de confiabilidade, controle de congestionamento, retransmissão e ordenação dos dados. Já o R-UDP foi implementado sobre UDP utilizando a estratégia Stop-and-Wait, em que cada pacote enviado precisa ser confirmado por um ACK antes do envio do próximo pacote.

2. Implementação

2.1. Transferência via TCP

A implementação TCP utilizou sockets do tipo `SOCK_STREAM`. O cliente estabelece uma conexão com o servidor, envia inicialmente o campo `X-Custom-Auth` e, em seguida, realiza a transferência do arquivo em blocos. O servidor recebe os dados e grava o arquivo localmente.

2.2. Transferência via R-UDP

A implementação R-UDP utilizou sockets do tipo `SOCK_DGRAM`. Para garantir confiabilidade, foram utilizados números de sequência, ACKs, timeout, retransmissão e checksum MD5 por bloco. O protocolo adotado foi Stop-and-Wait, portanto o cliente envia um pacote e aguarda o ACK correspondente antes de enviar o próximo.

O cabeçalho de cada pacote R-UDP possui:

- número de sequência: 10 bytes;
- checksum MD5: 32 bytes;
- dados do arquivo.

Além disso, antes da transferência, o cliente envia o campo `X-Custom-Auth`, contendo o hash SHA-256 da matrícula concatenada com o nome do aluno.

3. Ambiente de Testes

Os testes foram realizados utilizando Docker com dois containers: um para o cliente e outro para o servidor. A simulação de atraso e perda de pacotes foi feita com o comando `tc qdisc`, usando o módulo `netem`.

Foram utilizados três cenários:

Tabela 1. Cenários de teste

Cenário	Delay	Perda de pacotes
A	10 ms	0%
B	50 ms	5%
C	100 ms	10%

```
root@cd3b13377f3b:/app# tc qdisc add dev eth0 root netem delay 10ms loss 0%
root@cd3b13377f3b:/app# python servidor_tcp.py
Servidor TCP esperando conexão...
```

Figura 1. Configuração do Cenário A com 10 ms de delay e 0% de perda.

Ao finalizar os testes de um cenário, o comando `tc qdisc del dev eth0 root` foi utilizado para remover a configuração anterior. Em seguida, foi aplicada a configuração correspondente ao próximo cenário, permitindo a realização dos experimentos sob diferentes condições de rede.

```
root@cd3b13377f3b:/app# tc qdisc del dev eth0 root
root@cd3b13377f3b:/app# tc qdisc add dev eth0 root netem delay 50ms loss 5%
root@cd3b13377f3b:/app# python servidor_tcp.py
Servidor TCP esperando conexão...
```

Figura 2. Configuração do Cenário B com 50 ms de delay e 5% de perda.

```
root@cd3b13377f3b:/app# tc qdisc del dev eth0 root
root@cd3b13377f3b:/app# tc qdisc add dev eth0 root netem delay 100ms loss 10%
root@cd3b13377f3b:/app# python servidor_tcp.py
Servidor TCP esperando conexão...
```

Figura 3. Configuração do Cenário C com 100 ms de delay e 10% de perda.

4. Resultados

Foram realizadas 10 execuções para cada protocolo em cada cenário. Os resultados foram armazenados em arquivos CSV e processados em Python com Pandas e Matplotlib.

Tabela 2. Resultados estatísticos do throughput

Protocolo	Cenário	Tempo médio (s)	Mín. KB/s	Média KB/s	Máx. KB/s	Desvio
TCP	A	0.096	49860.54	53326.26	58435.09	2997.75
TCP	B	0.364	12558.83	14075.01	14306.72	536.74
TCP	C	0.730	6322.26	7033.17	7228.34	366.28
R-UDP	A	7.437	686.06	688.50	691.47	1.87
R-UDP	B	67.361	66.13	76.37	82.14	5.46
R-UDP	C	200.844	6.81	33.82	40.05	9.75

Tabela 3. Retransmissões médias do R-UDP

Cenário	Perda	Retransmissões médias
A	0%	0.0
B	5%	34.0
C	10%	73.9

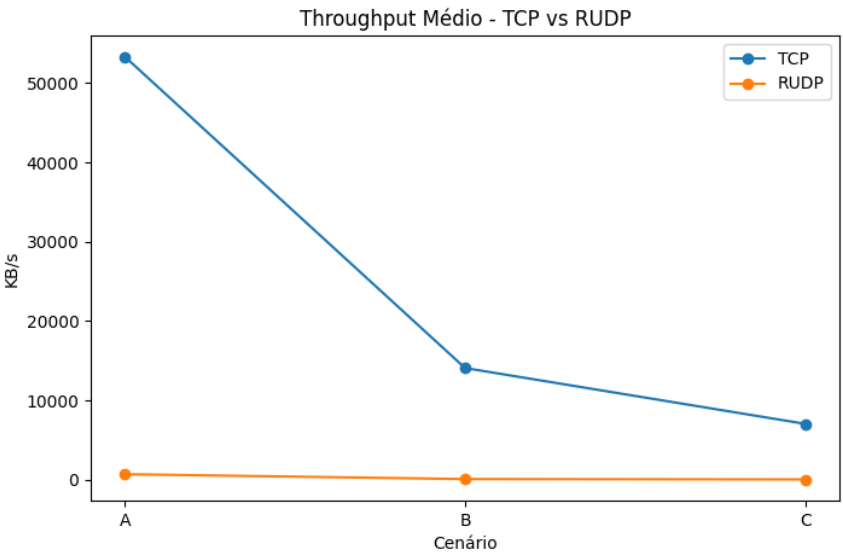


Figura 4. Throughput médio do TCP e do R-UDP nos três cenários.

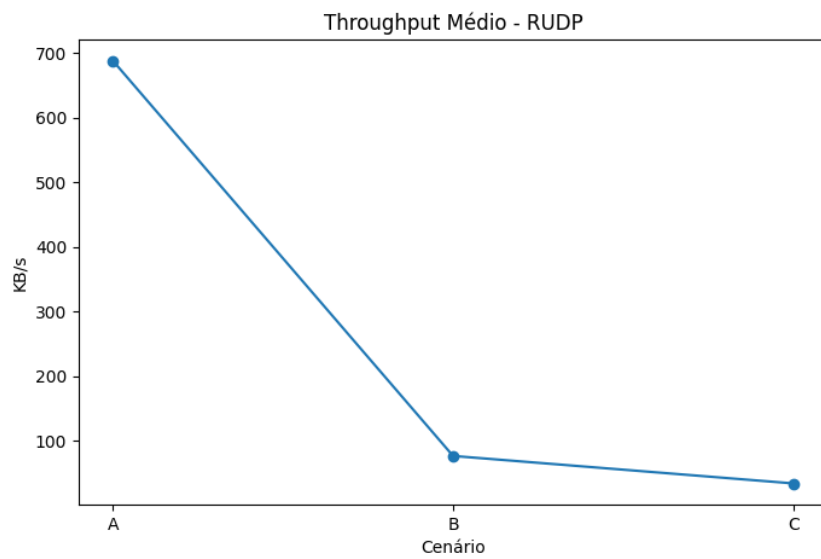


Figura 5. Throughput médio do R-UDP nos três cenários.

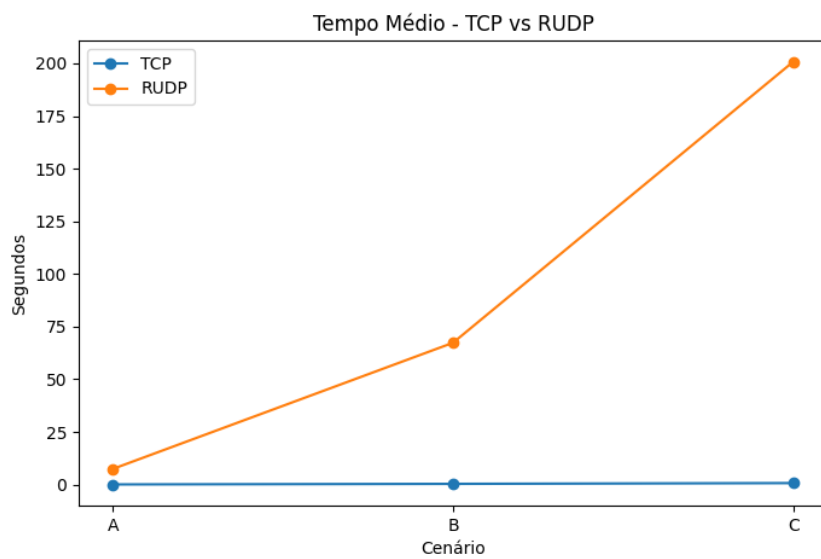


Figura 6. Tempo médio de transferência em cada cenário.

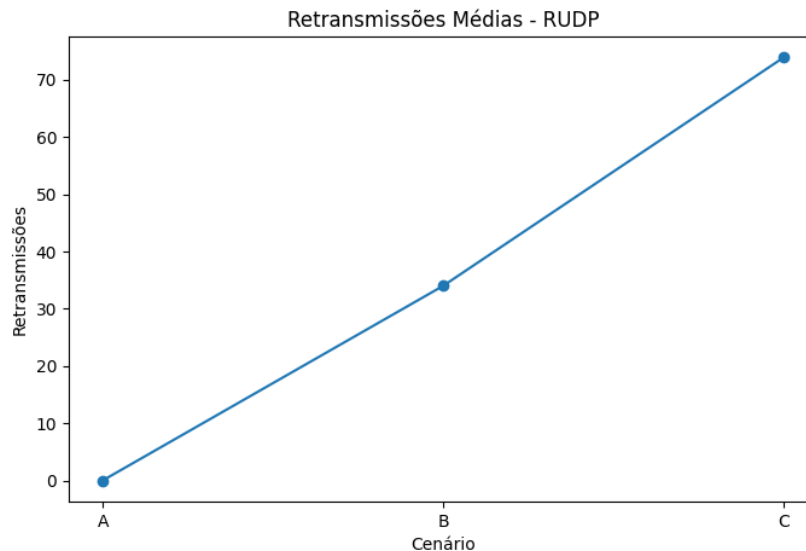


Figura 7. Quantidade média de retransmissões no R-UDP.

5. Discussão

Os resultados mostram que o TCP apresentou desempenho superior ao R-UDP em todos os cenários. No Cenário A, sem perda de pacotes, o TCP obteve throughput médio de aproximadamente 53326 KB/s, enquanto o R-UDP obteve cerca de 688 KB/s. Essa diferença ocorre porque o TCP trabalha de forma mais eficiente, enquanto o R-UDP implementado utiliza Stop-and-Wait, enviando apenas um pacote por vez.

No Cenário B, com 5% de perda e 50 ms de delay, o TCP caiu para aproximadamente 14075 KB/s. O R-UDP caiu para aproximadamente 76 KB/s, com média de 34 retransmissões. Isso mostra que a perda de pacotes afeta fortemente o protocolo implementado, pois cada perda força timeout e retransmissão.

No Cenário C, com 10% de perda e 100 ms de delay, a diferença ficou ainda mais evidente. O TCP obteve throughput médio de aproximadamente 7033 KB/s, enquanto o R-UDP caiu para aproximadamente 34 KB/s, com média de 73.9 retransmissões. Assim, o aumento da perda e do atraso prejudicou muito mais o R-UDP do que o TCP.

6. Validação e Overhead

O campo `X-Custom-Auth` foi implementado usando SHA-256 da matrícula concatenada com o nome do aluno. Esse campo permite identificar o tráfego da aplicação.

No R-UDP, cada pacote possui um cabeçalho de aplicação composto por 10 bytes para o número de sequência e 32 bytes para o checksum MD5. Portanto, o overhead fixo por pacote de dados é:

$$10 \text{ bytes} + 32 \text{ bytes} = 42 \text{ bytes}$$

Além disso, o campo `X-Custom-Auth` possui 64 bytes e é enviado no início da comunicação. Os ACKs também introduzem overhead adicional, pois cada pacote recebido corretamente gera uma confirmação enviada pelo servidor.

```
Esperando X-Custom-Auth...
X-Custom-Auth recebido:
b16de223936c74916a7b2d34505b5c418e44545cfc0e66926abf27feefea363d
Auth válido!
```

Figura 8. Servidor recebendo o campo X-Custom-Auth.

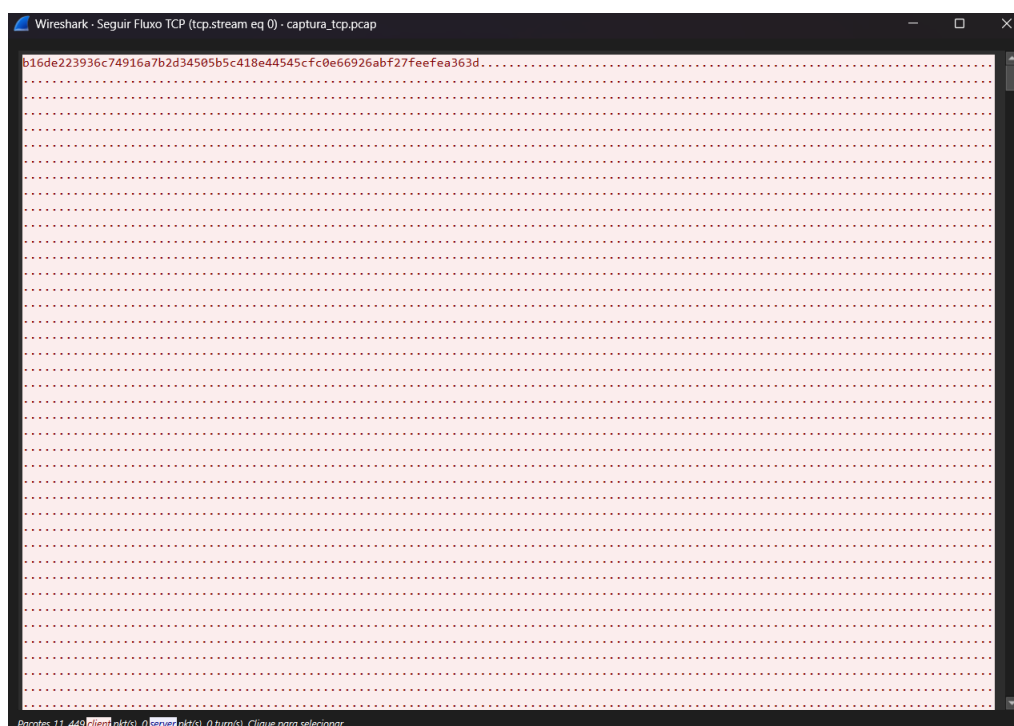


Figura 9. Captura do campo X-Custom-Auth utilizando o Wireshark e a funcionalidade Follow TCP Stream.

Além da validação realizada pelo servidor, o campo X-Custom-Auth também foi identificado na captura de rede realizada com TCPDump e analisada no Wireshark. A Figura 9 apresenta o fluxo TCP contendo o hash SHA-256 transmitido pelo cliente antes do início da transferência do arquivo.

7. Respostas às Perguntas Obrigatórias

7.1. 1. Como a curva de throughput do TCP se comportou em comparação ao R-UDP sob 10% de perda?

Sob 10% de perda, o TCP apresentou queda de desempenho, mas ainda manteve throughput médio de aproximadamente 7033 KB/s. O R-UDP, por outro lado, caiu para aproximadamente 34 KB/s. Isso ocorreu porque a implementação R-UDP usa Stop-and-Wait, ou seja, cada pacote precisa aguardar um ACK antes do envio do próximo. Quando há perda de pacotes ou de ACKs, o cliente precisa esperar o timeout para retransmitir, aumentando muito o tempo total de transferência.

7.2. 2. Qual foi o overhead introduzido pelos cabeçalhos do protocolo R-UDP?

O cabeçalho de cada pacote R-UDP possui 10 bytes para o número de sequência e 32 bytes para o checksum MD5, totalizando 42 bytes de overhead por pacote de dados. Além disso, o campo `X-Custom-Auth` possui 64 bytes e é enviado no início da comunicação. Também há overhead dos ACKs, pois o servidor envia uma confirmação para cada pacote recebido corretamente.

7.3. 3. Houve discrepância entre o tempo medido pelo Python e o registrado no Wireshark/TCPDump? Justifique.

A captura realizada com TCPDump e analisada no Wireshark confirmou a presença do campo `X-Custom-Auth` e o volume de tráfego correspondente à transferência realizada pela aplicação. Pequenas diferenças temporais podem ocorrer devido ao tempo de processamento da pilha de rede e da ferramenta de captura, porém os resultados observados foram compatíveis com os valores registrados pela aplicação.

8. Conclusão

Este trabalho permitiu comparar o comportamento do TCP nativo com uma implementação confiável sobre UDP. Os testes mostraram que o TCP apresentou desempenho muito superior em todos os cenários, principalmente em condições de perda e maior atraso. O R-UDP implementado conseguiu garantir a entrega por meio de ACKs, timeout, retransmissão e checksum, mas apresentou desempenho inferior devido ao uso do método Stop-and-Wait.

Conclui-se que, embora seja possível implementar confiabilidade sobre UDP, uma abordagem simples como Stop-and-Wait sofre bastante em redes com perda e latência. Para melhorar o desempenho do R-UDP, seria possível implementar janelas deslizantes, como Go-Back-N ou Selective Repeat.